

Bank Customers Churn Prediction

Data Science Mini Project

Student Name: Nirmala G

Batch: DSB02

Submission Date: 26 July 2026

1. Problem Definition

1.1 Business / Real-World Problem Statement

Customer churn is a critical challenge for banking institutions as acquiring new customers costs significantly more than retaining existing ones. When customers close their accounts or switch to competitor banks, it leads to direct revenue loss and reduced customer lifetime value. Identifying customers who are likely to churn enables banks to take targeted retention measures, offer proactive loyalty incentives, and improve overall customer satisfaction.

1.2 Project Objectives

- **Predict Customer Churn:** Develop a binary classification Machine Learning model to accurately predict whether a bank customer will churn (1) or stay (0).
- **Identify Key Risk Factors:** Analyze demographic, financial, and transactional features (such as credit score, balance, age, number of products, and activity status) to identify key drivers behind customer churn.
- **Build an End-to-End Pipeline:** Perform thorough data preprocessing, feature engineering, model evaluation, and hyperparameter tuning to ensure optimal model performance.
- **Model Deployment:** Deploy the final model as an interactive web application to allow real-time prediction for bank officers and risk analysts.

1.3 Machine Learning Problem Type

Supervised Learning – Classification

Explanation: This is a Supervised Learning Binary Classification task because the dataset contains labelled output targets where we predict whether a customer will leave the bank (1 = Churn) or stay (0 = Not Churned).

2. Dataset Understanding

2.1 Dataset Source

Dataset Source Link: [OpenML - Bank Customer Churn Dataset](#)

Citation: OpenML / Churn-for-Bank-Customers

2.2 Dataset Description

The dataset used in this project contains historical information about bank customers to analyze and predict customer churn behaviour.

- Number of rows: [10,000]
- Number of columns: [12]
- Dataset purpose: To identify key behavioural and demographic factors that influence customer churn, enabling the bank to build proactive customer retention strategies
- Data collection source: OpenML / Churn-for-Bank-Customers

2.3 Feature Description

Feature Name	Data Type	Description
CustomerId	Integer	Unique identifier for each customer.
CreditScore	Integer	Credit score of the customer, reflecting financial trustworthiness.
Country	String	Country/Location of the customer (e.g., France, Spain, Germany).
Gender	String	Gender of the customer (Male/Female).
Age	Integer	Age of the customer.
Tenure	Integer	Number of years the customer has been with the bank.
Balance	Float	Total account balance available in the customer's account.
NumOfProducts	Integer	Number of bank products the customer actively uses.
HasCrCard	Integer	Indicates whether the customer holds a credit card (1 = Yes, 0 = No).
IsActiveMember	Integer	Indicates whether the customer is an active bank member (1 = Yes, 0 = No).
EstimatedSalary	Float	Estimated annual salary of the customer.
Churn (Target)	Integer	Target variable indicating customer churn (1 = Churned, 0 = Retained).

2.4 Target Variable

Name: Churn

Description: A binary categorical variable indicating whether a bank customer has left (churned) or stayed with the bank (1 = Churned / Left, 0 = Retained / Stayed).

Prediction Goal: To accurately predict whether a customer is likely to close their bank account in the near future, allowing the bank to take proactive customer retention actions.

3. Exploratory Data Analysis (EDA)

3.1 Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from xgboost import XGBClassifier
from imblearn.over_sampling import SMOTE
from sklearn.metrics import accuracy_score, f1_score,
confusion_matrix, classification_report, precision_score, recall_score
```

3.2 Load Dataset

```
df=pd.read_csv("Bank Customer Churn Prediction.csv")

df.head()
```

	customer_id	credit_score	country	gender	age	tenure	balance	products_number	credit_card	active_member	estimated_salary	churn
0	15634602	619	France	Female	42	2	0.00	1	1	1	101348.88	1
1	15647311	608	Spain	Female	41	1	83807.86	1	0	1	112542.58	0
2	15619304	502	France	Female	42	8	159660.80	3	1	0	113931.57	1
3	15701354	699	France	Female	39	1	0.00	2	0	0	93826.63	0
4	15737888	850	Spain	Female	43	2	125510.82	1	1	1	79084.10	0

3.3 Dataset Overview

```
df.shape
```

✓ 0.0s

(10000, 12)

Python

Dataset Size: The dataset contains 10,000 rows and 12 columns.

Renaming column names to improve readability

```
df=df.rename(columns={
    'credit_score':'Credit_Score',
    'country':'Country',
    'gender':'Gender',
    'tenure':'Tenure',
    'products_number':'NumOfProducts',
    'credit_card':'HasCreditCard',
    'active_member':'IsActiveMember',
    'estimated_salary':'EstimatedSalary',
    'churn':'Churn',
    'balance':'Balance',
    'age':'Age'
})
```

✓ 0.0s

Python

```
df.columns
```

✓ 0.0s

```
Index(['customer_id', 'Credit_Score', 'Country', 'Gender', 'Age', 'Tenure',
      'Balance', 'NumOfProducts', 'HasCreditCard', 'IsActiveMember',
      'EstimatedSalary', 'Churn'],
      dtype='str')
```

Python

Column Names: Column names were checked and found to be properly formatted and self-explanatory.

```
df.info()
0.0s Python

<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customer_id           10000 non-null  int64
1   Credit_Score           10000 non-null  int64
2   Country                10000 non-null  str
3   Gender                 10000 non-null  str
4   Age                   10000 non-null  int64
5   Tenure                 10000 non-null  int64
6   Balance                10000 non-null  float64
7   NumOfProducts          10000 non-null  int64
8   HasCreditCard          10000 non-null  int64
9   IsActiveMember         10000 non-null  int64
10  EstimatedSalary         10000 non-null  float64
11  Churn                   10000 non-null  int64
dtypes: float64(2), int64(8), str(2)
memory usage: 937.6 KB
```

Information: All 12 columns have 10000 non-null values, meaning there are no missing values in the dataset.

```
df.dtypes
0.0s Python

customer_id      int64
Credit_Score     int64
Country          str
Gender           str
Age              int64
Tenure           int64
Balance          float64
NumOfProducts    int64
HasCreditCard    int64
IsActiveMember   int64
EstimatedSalary  float64
Churn            str
dtype: object
```

Data Types: Includes integer (int64), float (float64), and string (str/object) variables.

Target Variable: Churn is the target column representing customer retention.

Statistical Summary (df.describe())

```
df.describe()
0.0s Python
```

	Credit_Score	Age	Tenure	Balance	NumOfProducts	HasCreditCard	IsActiveMember	EstimatedSalary
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	650.528800	38.921800	5.012800	76485.889288	1.530200	0.70550	0.515100	100090.239881
std	96.653299	10.487806	2.892174	62397.405202	0.581654	0.45584	0.499797	57510.492818
min	350.000000	18.000000	0.000000	0.000000	1.000000	0.000000	0.000000	11.580000
25%	584.000000	32.000000	3.000000	0.000000	1.000000	0.000000	0.000000	51002.110000
50%	652.000000	37.000000	5.000000	97198.540000	1.000000	1.000000	1.000000	100193.915000
75%	718.000000	44.000000	7.000000	127644.240000	2.000000	1.000000	1.000000	149388.247500
max	850.000000	92.000000	10.000000	250898.090000	4.000000	1.000000	1.000000	199992.480000

Observations:

- Customer Age: Ranges from 18 to 92 years, with an average customer age of ~39 years.
- Credit Score: Ranges between 350 and 850, averaging around 650.
- Account Balance: Ranges from 0 to over 2,50,000, with a mean balance of ~76,485.
- Tenure & Products: Customers stay an average of 5 years with the bank and hold 1 to 2 products on average.

3.4 Missing Value Analysis

```
df.isnull().sum()
✓ 0.0s Python
```

customer_id	0
Credit_Score	0
Country	0
Gender	0
Age	0
Tenure	0
Balance	0
NumOfProducts	0
HasCreditCard	0
IsActiveMember	0
EstimatedSalary	0
Churn	0

dtype: int64

There are no missing values identified in the dataset.

All features are fully populated, which ensures data completeness and allows us to proceed directly to model training without any imputation.

3.5 Duplicate Value Analysis

```
df.duplicated().sum()
✓ 0.1s Python
```

np.int64(0)

A check for duplicate rows returned a count of zero, confirming that every record in the dataset represents a unique customer. The absence of duplicate data ensures that our statistical analysis and future model predictions will not be biased by redundant information.

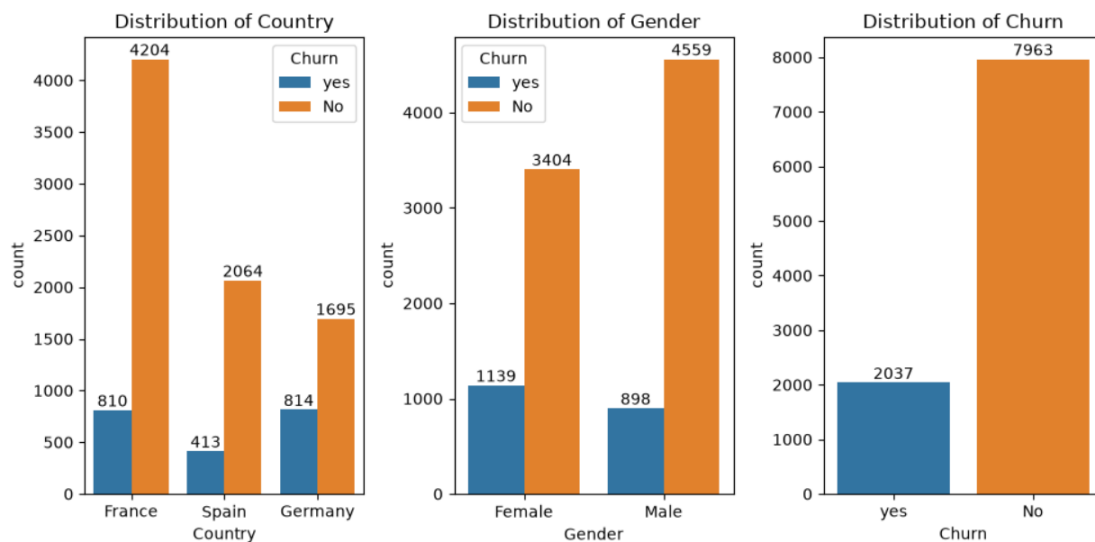
3.6 Visualizations

Visualization 1: Univariate analysis of Categorical columns

```
fig, axes = plt.subplots(ncols=3, nrows=1, figsize=(10, 5))
for i, col in enumerate(Categorical_columns):
    sns.countplot(data=df, x=col, hue='Churn', ax=axes[i]).set_title(f'Distribution of {col}')
    for container in axes[i].containers:
        axes[i].bar_label(container)
plt.tight_layout()
plt.show()
```

✓ 1.5s

Python



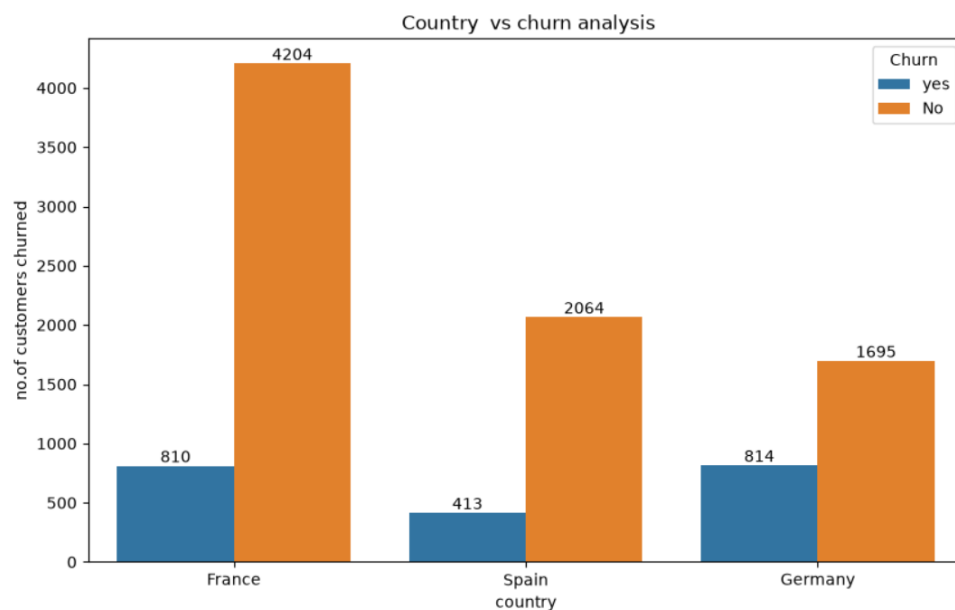
Distribution of Churn (Target Variable)

- Observation: The dataset is highly imbalanced.
- Out of the total customers, 7,963 customers have stayed with the bank ("No" Churn),
- while 2,037 customers have left the bank ("yes" Churn).
- Insight: The overall churn rate is approximately 20.37%. This serves as a baseline value for our predictive models.

Visualization 2: Country vs Churn Analysis

```
plt.figure(figsize=(10,6))

ax=sns.countplot(data=df,x='Country',hue="Churn")
plt.bar_label(ax.containers[0])
plt.bar_label(ax.containers[1])
plt.title("Country vs churn analysis")
plt.xlabel("country")
plt.ylabel("no.of customers churned")
plt.show()
```



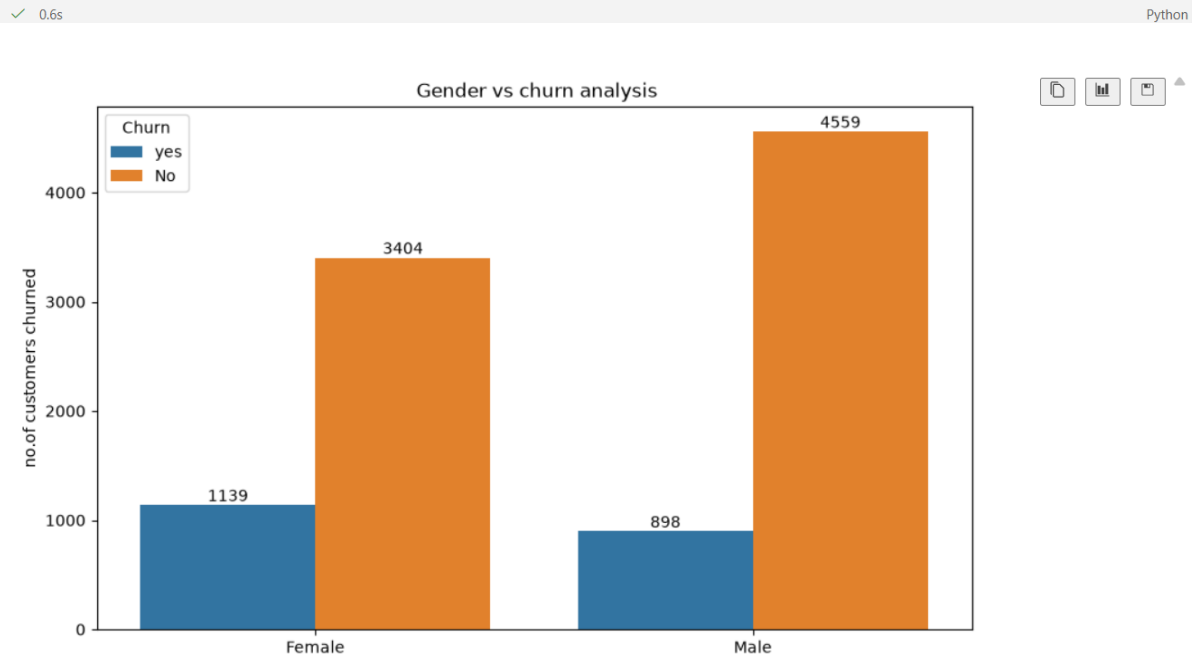
Country vs. Churn Analysis:

- France & Spain: France has the highest number of customers overall. Both France and Spain show relatively low and stable churn rates
- Germany (Critical Risk): Germany has a significantly high churn rate. Despite having fewer total customers than France, it has the highest absolute number of churned customers (814), meaning roughly 32% of German customers are leaving.
- Action Item: The business should prioritize investigating the German market to understand and fix the high customer loss.

Visualization 3: Gender vs Churn Analysis

```
plt.figure(figsize=(10,6))

ax=sns.countplot(data=df,x='Gender',hue="Churn")
plt.bar_label(ax.containers[0])
plt.bar_label(ax.containers[1])
plt.title(" Gender vs churn analysis")
plt.xlabel("Gender")
plt.ylabel("no.of customers churned")
plt.show()
```



Gender vs. Churn Analysis:

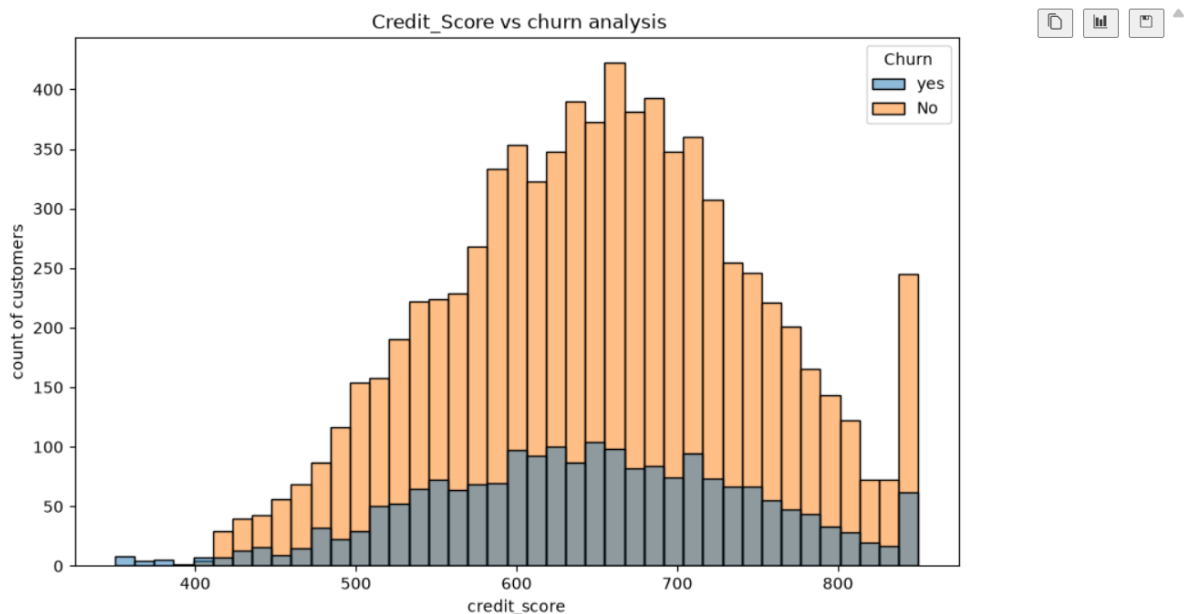
- Higher Risk in Females: Female customers show a significantly higher tendency to leave the bank.
- A total of 1,139 females has churned compared to only 898 males, despite there being fewer female customers overall in the dataset.
- Higher Loyalty in Males: Male customers show much stronger retention, with 4,559 staying with the bank, resulting in a notably lower churn rate compared to females.
- Business Takeaway: There is a clear gender gap in customer retention.
- The business needs to investigate why female customers are dissatisfied—potentially looking into product preferences, targeted customer service, or specialized financial features to improve female retention.

Visualization 4: Credit score vs Churn

```
plt.figure(figsize=(10,6))
ax=sns.histplot(data=df,hue='Churn',x='Credit_Score')
plt.title(" Credit_Score vs churn analysis")
plt.xlabel("credit_score")
plt.ylabel("count of customers")
plt.show()
```

✓ 0.6s

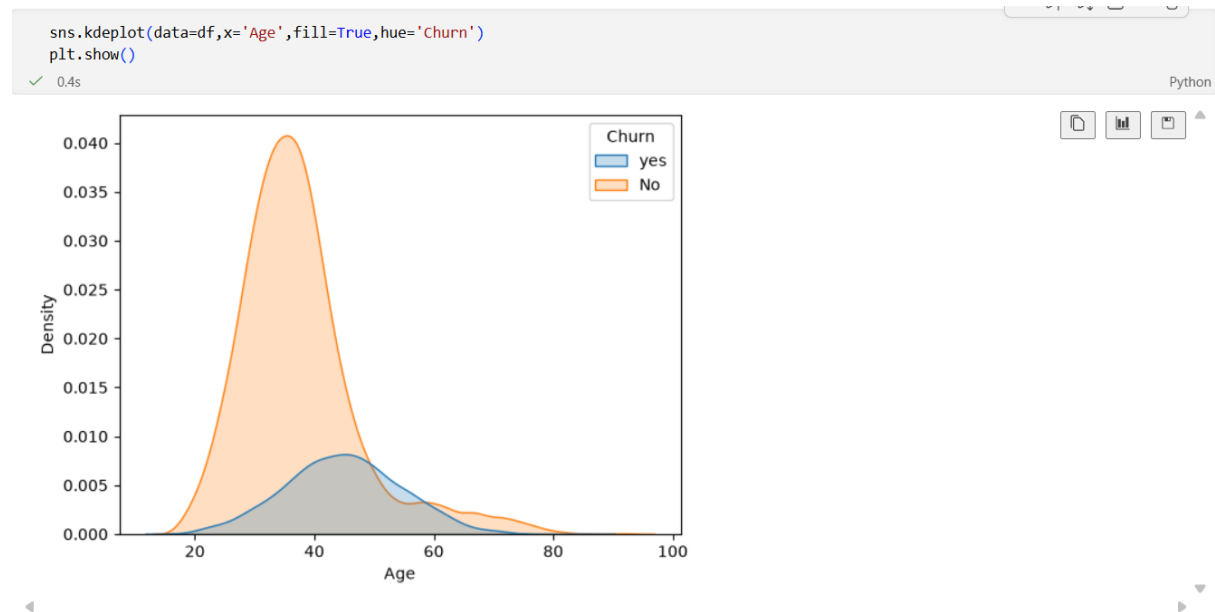
Python



Observations:

- The histogram illustrates a normal distribution of customer credit scores centered between 600 and 700.
- When evaluating the churn status (Yes/No), the ratio of churned customers remains visually consistent across all credit score brackets.
- This indicates that credit score is not a primary driving factor for customer churn in this dataset, as high-score individuals leave the bank at a proportional rate to low-score individuals.

Visualization 5: Age vs Churn



Observations:

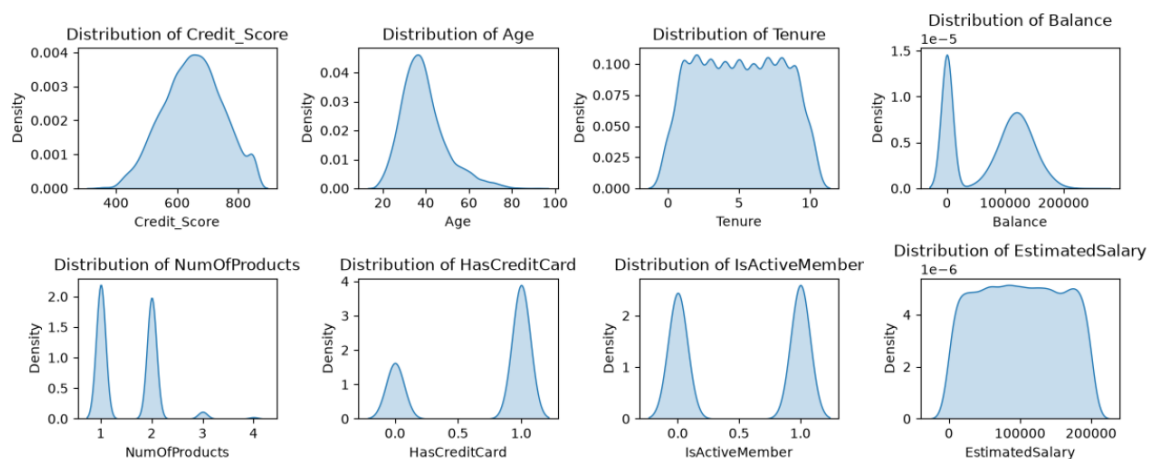
- Younger customers (aged 20-40) are highly loyal and prefer to stay with the bank.
- Middle-aged customers (aged 40-50) show the highest rate of leaving (churning) the bank.
- Therefore, Age is a strong predictor of churn, and the bank should focus its retention strategies on the 40-50 age group.

Visualization 6: Univariate Analysis of Numerical Columns

```
fig, axes = plt.subplots(ncols=4, nrows=2, figsize=(12, 5))
ax = axes.flatten()
for i, col in enumerate(Numerical_columns):
    sns.kdeplot(data=df, x=col, ax=ax[i], fill=True).set_title(f'Distribution of {col}')
plt.tight_layout()
plt.show()
```

✓ 2.9s

Python



Univariate Analysis Insights:

- **Credit_Score:** Shows a roughly normal (bell-shaped) distribution centered around 600–650, with a slight left skew (a few customers have exceptionally low credit scores).
- **Age:** Right-skewed distribution. The majority of the bank's customers are young to middle-aged adults, concentrated between 30 and 45 years old, with a long tail extending into elderly age groups.
- **Tenure:** Shows a uniform distribution. Customers are evenly distributed across all tenure lengths (from 0 to 10 years), indicating steady customer acquisition over the years.
- **Balance:** This feature is strictly bimodal. There is a huge spike at 0, showing a massive segment of customers with empty or inactive balances. The remaining active balances are normally distributed around 100,000 to 125,000.
- **NumOfProducts:** Highly discrete distribution. The vast majority of customers only purchase 1 or 2 products from the bank. Extremely few customers hold 3 or 4 products.

- **HasCreditCard and IsActiveMember:**

These are binary/categorical flags represented numerically:

HasCreditCard: Strongly skewed toward 1.0. A significant majority of the bank's customers own a credit card.

IsActiveMember: Fairly evenly split between active (1.0) and inactive (0.0) members, though active members have a slightly higher peak.

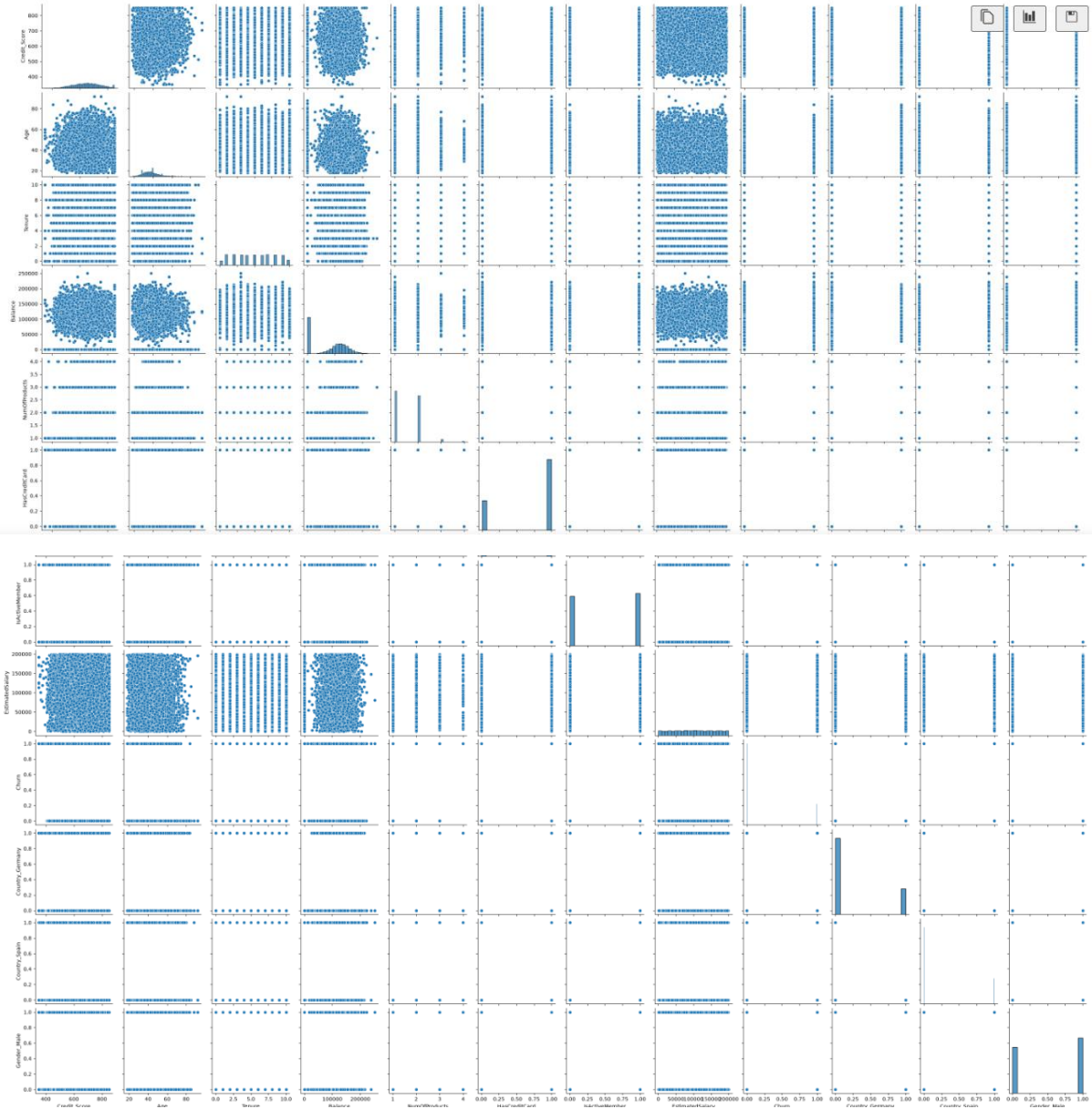
- **EstimatedSalary:** Completely uniform distribution spanning from 0 to 200,000. This implies that the salary values are evenly distributed across the customer base, which is typical for synthetic or uniformly sampled income fields.

Visualization 7: Pair plot

```
sns.pairplot(data=df)  
plt.show()
```

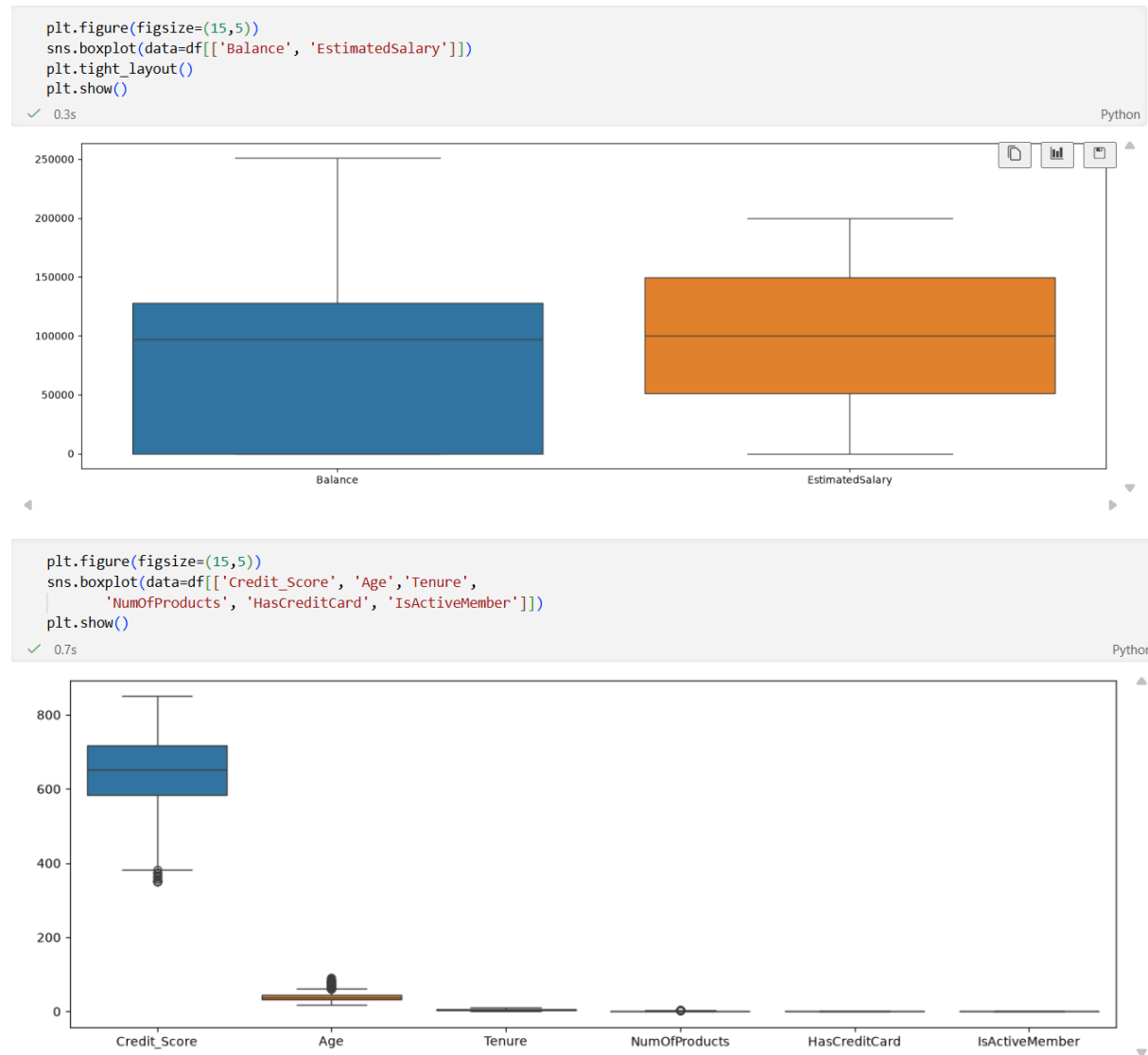
✓ 17.8s

Pytl



4. Data Preprocessing

4.1 Outlier Detection and Treatment



Outlier Treatment & Insights:

- **Age (Slight Correlation):** Age shows a minor correlation with customer churn. The older age outliers represent genuine senior citizen profiles, making their retention valuable for capturing this specific age-group behaviour.
- **Credit Score (No Impact on Churn):** Analysis reveals that Credit_Score does not significantly impact churn, as customers with both high and low credit scores are churning uniformly.

- **Conclusion:** Because these outliers represent valid real-world data points and their distribution aligns with the overall churn patterns, no outliers will be removed to avoid loss of organic data.

4.2 One Hot Encoding

```
encoder=OneHotEncoder()
```

✓ 0.0s Python

```
Categorical_columns
```

✓ 0.0s Python

```
['Country', 'Gender', 'Churn']
```

```
df=pd.get_dummies(df,columns=["Country","Gender"],dtype=int,drop_first=True)
```

✓ 0.0s Python

Age	Tenure	Balance	NumOfProducts	HasCreditCard	IsActiveMember	EstimatedSalary	Churn	Country_Germany	Country_Spain	Gender_Male
42	2	0.00	1	1	1	101348.88	yes	0	0	0
41	1	83807.86	1	0	1	112542.58	No	0	1	0
42	8	159660.80	3	1	0	113931.57	yes	0	0	0
39	1	0.00	2	0	0	93826.63	No	0	0	0
43	2	125510.82	1	1	1	79084.10	No	0	1	0
...	...	...	...	...	...	...	...	...	...	...
39	5	0.00	2	1	0	96270.64	No	0	0	1
35	10	57369.61	1	1	1	101699.77	No	0	0	1
36	7	0.00	1	0	1	42085.58	yes	0	0	0
42	3	75075.31	2	1	0	92888.52	yes	1	0	1
28	4	130142.79	1	1	0	38190.78	No	0	0	0

Observation:

Using `pd.get_dummies()` with `drop_first=True` successfully converts categorical variables (Country and Gender) into numerical binary format. This effectively prevents the dummy variable trap (multicollinearity), ensuring optimal stability and performance for machine learning models.

4.3 Feature Scaling

Standardization

```
scaler=StandardScaler()
```

✓ 0.0s

Python

```
Numerical_columns
```

✓ 0.0s

Python

```
['Credit_Score',  
'Age',  
'Tenure',  
'Balance',  
'NumOfProducts',  
'HasCreditCard',  
'IsActiveMember',  
'EstimatedSalary']
```

```
x_train[Numerical_columns]=scaler.fit_transform(x_train[Numerical_columns])
```

✓ 0.0s

Python

```
x_test[Numerical_columns]=scaler.transform(x_test[Numerical_columns])
```

✓ 0.0s

Python

Age	Tenure	Balance	NumOfProducts	HasCreditCard	IsActiveMember	EstimatedSalary	Churn	Country_Germany	Country_Spain	Gender_Male
42	2	0.00	1	1	1	101348.88	1	0	0	0
41	1	83807.86	1	0	1	112542.58	0	0	1	0
42	8	159660.80	3	1	0	113931.57	1	0	0	0
39	1	0.00	2	0	0	93826.63	0	0	0	0
43	2	125510.82	1	1	1	79084.10	0	0	1	0
...	...	...	...	...	...	...	...	...	...	...
39	5	0.00	2	1	0	96270.64	0	0	0	1
35	10	57369.61	1	1	1	101699.77	0	0	0	1
36	7	0.00	1	0	1	42085.58	1	0	0	0
42	3	75075.31	2	1	0	92888.52	1	1	0	1
28	4	130142.79	1	1	0	38190.78	0	0	0	0

Conclusion:

Feature scaling via StandardScaler standardizes all numerical features to have a mean of 0 and a variance of 1, preventing high-magnitude columns (like Balance or EstimatedSalary) from dominating distance-based algorithms. After scaling, all continuous features share a uniform scale, ensuring faster model convergence and unbiased feature importance.

4.4 Train-Test Split

```
from sklearn.model_selection import train_test_split
```

✓ 0.5s Python

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.25,random_state=42)
```

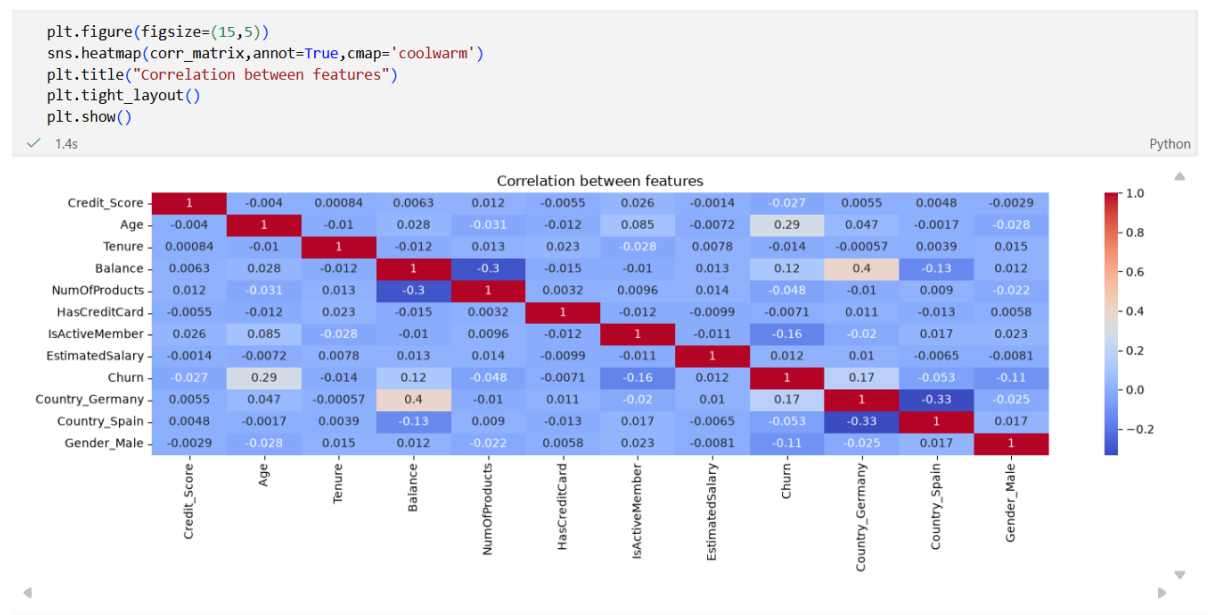
✓ 0.0s Python

Conclusion:

The dataset was split into training (75%) and testing (25%) subsets using `train_test_split` with `random_state=42` to ensure reproducible results. This separation allows the model to learn patterns from the training data while retaining unseen test data to evaluate its actual performance and prevent overfitting.

5. Feature Engineering & Feature Selection

5.1 Correlation Analysis



Based on the correlation heatmap:

Weak Linear Correlations: Most independent features (like Credit_Score, EstimatedSalary, Gender_Male, and Tenure) show a correlation value very close to 0 with the target variable Churn. This indicates that there are no strong, straightforward linear relationships between these individual features and customer churn.

Top Contributing Indicators: The features showing noticeable linear trends with churn are Age (0.29) (older customers are more likely to churn), Country_Germany (0.17) (customers in Germany are more likely to churn), IsActiveMember (-0.16) (active members are less likely to churn), and Balance (0.12). Additionally, Country_Germany shows a moderate positive correlation with Balance (0.40).

We Must Include All Features: Just because a feature has a correlation close to 0 doesn't mean it is useless. Correlation only measures linear relationships.

In a real-world banking scenario, customer behaviour is complex. For example, a low credit score might not cause churn on its own, but a customer with a low credit score combined with a high balance and low tenure might be at high risk.

Final Strategy: To capture these complex, hidden interactions and geographic patterns, we will include all features in our dataset—including the newly created One-Hot Encoded features—rather than dropping the low-correlation ones.

5.2 Feature Selection & Justification:

Low Linear Correlation: The correlation heatmap indicates that most independent features have weak linear relationships with the target variable Churn (with Age having the maximum correlation of 0.29).

Non-Linear Interactions: Since customer churn is a complex behavioural pattern, features with low linear correlation can still possess strong non-linear relationships and interactions that simple correlation cannot capture.

Multicollinearity Check: There is no significant multicollinearity between the independent variables, meaning every feature provides unique information to the dataset.

Final Decision: Dropping features based purely on low correlation might lead to a loss of critical data signals. Therefore, all features are retained to allow robust tree-based algorithms (like Random Forest or XGBoost) to effectively model the underlying non-linear patterns.

6. Model Building

6.1 Baseline Model

To systematically evaluate multiple machine learning algorithms, a dictionary of candidate classifiers was iterated using a for loop. This approach allows automated training and benchmark evaluation across all models on the same training/testing split.

Key Strategy & Observation

Metrics Tracked: For each model in the loop, Accuracy, F1-Score, Confusion Matrix, and a detailed Classification Report were generated and stored in a dictionary.

Goal: The primary goal of this comparative baseline evaluation was to benchmark default performances across algorithms and identify the top-performing model(s) based on F1-score and Recall (vital for handling class imbalance in churn prediction).

Next Steps: The highest-performing model identified from this automated comparison was selected as the candidate for further hyperparameter tuning to optimize final predictive performance.

```
classifiers={
    'LogisticRegression':LogisticRegression(class_weight='balanced'),
    'KNeighborsClassifier':KNeighborsClassifier(),
    'RandomForestClassifier':RandomForestClassifier(random_state=42,class_weight='balanced'),
    'svm_classifier':SVC(),
    'DecisionTreeClassifier':DecisionTreeClassifier(),
    'GaussianNB':GaussianNB(),
    'XGBClassifier':XGBClassifier(scale_pos_weight=4)
}
```

✓ 0.0s Python

Since the dataset is imbalanced, setting `class_weight='balanced'` (and `scale_pos_weight` in XGBoost) penalizes misclassifications of the minority class more heavily. This forces the model to give equal importance to both classes during training, improving its ability to detect the minority class.

```
for name,clf in classifiers.items():
    clf.fit(x_train,y_train)
    y_pred=clf.predict(x_test)
    accuracy=accuracy_score(y_test,y_pred)
    f1=f1_score(y_test,y_pred)
    report=classification_report(y_test,y_pred)
    matrix=confusion_matrix(y_test,y_pred)

    result[name]={'Accuracy':accuracy,'f1 score': f1}
    print(f'{name} \n { classification_report(y_test,y_pred)} \n')
    print(f'{name} \n { confusion_matrix(y_test,y_pred)} \n')

result_df=pd.DataFrame(result).T
```

Observations:

Model	Accuracy	Churn Precision	Churn Recall	Churn F1-Score	Status / Conclusion
Logistic Regression	0.71	0.38	0.71	0.49	High Recall, but lower Precision
KNeighborsClassifier	0.84	0.66	0.44	0.53	Good Accuracy, but misses churners (Low Recall)
RandomForestClassifier	0.85	0.62	0.59	0.61	Balanced performance (Top Candidate)
SVM Classifier (SVC)	0.86	0.8	0.38	0.51	High Precision, but very low Recall
DecisionTreeClassifier	0.79	0.48	0.5	0.49	Moderate performance across all metrics
GaussianNB	0.83	0.63	0.36	0.46	Lowest Churn Recall
XGBClassifier	0.83	0.57	0.64	0.6	High Recall & Strong F1-Score (Top Candidate)

Detailed Analysis & Insights

- Because your target dataset is imbalanced (as indicated use of `class_weight='balanced'` and `scale_pos_weight=4`), Accuracy alone is misleading.
- For instance, SVM achieves the highest accuracy (85.76%), but its F1-Score drops to 0.5137, meaning it heavily predicts the majority (negative) class and misses many minority (positive) churn cases.

Top Performing Models:

- Random Forest and XGBoost are best performing.
- Random Forest Classifier: Achieves 84.72% Accuracy and 0.6062 F1-Score. Handling class imbalance via `class_weight='balanced'` allowed it to maintain high precision and recall simultaneously.
- XGBoost Classifier: Achieves 83.08% Accuracy and 0.6021 F1-Score. Setting `scale_pos_weight=4` significantly boosted its ability to capture the minority class.

Underperforming Models:

- Logistic Regression: Shows a relatively low accuracy (70.96%) and F1-score (0.4916), indicating that customer churn in this dataset has non-linear feature relationships.
- Gaussian Naive Bayes: While accuracy seems decent (83.04%), the lowest F1-score (0.4578) reveals poor recall on the target class

Conclusion & Next Steps:

Based on the baseline model comparison, RandomForestClassifier (F1-score: 0.61) and XGBClassifier (F1-score: 0.60, Recall: 0.64) outperformed other algorithms in effectively identifying potential churn customers. Consequently, Random Forest and XGBoost are selected as the top candidate models for Hyperparameter Tuning to further optimize model accuracy and F1-score.

6.2 Hyperparameter Tuning for RandomForestClassifier

```
model=RandomForestClassifier(class_weight='balanced')
✓ 0.0s Python
```

```
param_grid={
    'n_estimators': [50,100,200],
    'max_depth': [10,20,30],
    'min_samples_split': [2,5,10]
}
✓ 0.0s Python
```

```
from sklearn.model_selection import GridSearchCV
✓ 0.0s Python
```

```
grid_search=GridSearchCV(model,param_grid,scoring='f1',cv=3,n_jobs=-1,verbose=2)
✓ 0.0s Python
```

```
grid_search.fit(x_train,y_train)
✓ 3m 30.5s Python
```

```
best_random_forest=grid_search.best_estimator_
✓ 0.5s Python
```

Observation:

Model Stage	Accur acy	Churn Precision	Churn Recall	Churn F1- Score	Key Improvement
Baseline Random Forest	0.85	0.62	0.59	0.61	Initial benchmark model
Tuned Random Forest	0.84	0.6	0.64	0.62	Higher Recall (+5%) & Better F1-Score

Impact of Tuning:

Improved Churn Detection: Hyperparameter tuning using GridSearchCV significantly increased the Recall for the churn class from 0.59 to 0.64 (a 5% improvement), resulting in 316 correctly identified churned customers out of 497.

Optimal Trade-off: While overall accuracy slightly adjusted from 85% to 84.36%, the overall F1-Score improved to 0.62 (0.6178). In churn prediction, capturing more true churners (higher Recall) takes priority over marginal accuracy loss, making the Tuned Random Forest Model substantially more effective for real-world business retention strategies.

6.3 SMOTE (Synthetic Minority Over-sampling Technique)

To address class imbalance, SMOTE was implemented to generate synthetic samples for the minority churn class during training, with the goal of enhancing the model's ability to detect churners.

```
smote=SMOTE(random_state=42)
x_train_resampled,y_train_resampled=smote.fit_resample(x_train,y_train)
model1=RandomForestClassifier(max_depth= 20,min_samples_split=2, n_estimators= 100)
model1.fit(x_train_resampled,y_train_resampled)
y_pred_resampled=model1.predict(x_test)
accuracy=accuracy_score(y_test,y_pred_resampled)
```

Observations:

Model Scenario	Accuracy	Churn Precision (Class 1)	Churn Recall (Class 1)	Churn F1-Score (Class 1)	True Positives (Caught Churns)
Tuned Random Forest (No SMOTE)	84.36%	0.6	0.64 (64%)	0.62	316 / 497
Tuned Random Forest (+ SMOTE)	83.00%	0.57	0.61 (61%)	0.59	304 / 497

Evaluating SMOTE Impact:

Applying SMOTE oversampling prior to training with tuned Random Forest hyperparameters yielded a slightly lower overall F1-score (0.59 vs. 0.62) and Recall (0.61 vs. 0.64) compared to the tuned model without resampling.

Technical Insight:

This reduction indicates that synthetic oversampling introduced minor noise into the decision boundary, slightly degrading the precision and overall classification accuracy for this specific dataset.

Conclusion on Resampling:

This indicates that synthetic oversampling introduced minor noise into the decision boundary, confirming that algorithmic tuning without SMOTE performs better for this dataset.

6.4 Transition to XGBoost Tuning:

"Since XGBoost Classifier emerged as one of the top-performing algorithms in our baseline evaluation, Hyperparameter Tuning (via GridSearchCV / RandomizedSearchCV) was conducted on XGBoost to explore if further parameter optimization could outperform the tuned Random Forest model."

```
xg_new=XGBClassifier()
```

✓ 0.2s Python

```
param_grid={
    'learning_rate':[0.01,0.05,0.1,0.2],
    'n_estimators':[100,200,300],
    'max_depth':[3,5,7],
    'min_child_weight':[1,3,5],
    'scale_pos_weight':[1,3,4,5]
}
```

✓ 0.0s Python

```
grid_search=GridSearchCV(estimator=xg_new,param_grid=param_grid,scoring='f1',cv=5,n_jobs=-1,verbose=1)
```

✓ 0.3s Python

```
grid_search.fit(x_train,y_train)
```

✓ 2m 50.3s Python

Fitting 5 folds for each of 432 candidates, totalling 2160 fits

```
> GridSearchCV ⓘ ⓘ
> best_estimator_: XGBClassifier
  > XGBClassifier ⓘ
```

```
best_xgb=grid_search.best_estimator_
```

✓ 0.0s Python

```
y_pred=best_xgb.predict(x_test)
```

✓ 0.8s Python

```
grid_search.best_params_
```

✓ 0.0s Python

```
{'learning_rate': 0.05,
 'max_depth': 7,
 'min_child_weight': 1,
 'n_estimators': 100,
 'scale_pos_weight': 3}
```

```
accuracy=accuracy_score(y_test,y_pred)
report=classification_report(y_test,y_pred)
f1=f1_score(y_test,y_pred)
matrix=confusion_matrix(y_test,y_pred)
```

✓ 0.2s Python

```
print("hyperparameter tuned xgboost classifier")
print("Accuracy :",accuracy)
print("F1 score :",f1)
print(f'Confusion matrix : \n {matrix}')
print(f'Classification report : \n {report}')
```

✓ 0.5s Python

```
hyperparameter tuned xgboost classifier
Accuracy : 0.8344
F1 score : 0.6208791208791209
Confusion matrix :
[[1747 256]
 [ 158 339]]
Classification report :
      precision    recall  f1-score   support

     0       0.92      0.87      0.89       2003
     1       0.57      0.68      0.62        497

 accuracy          0.83       2500
 macro avg         0.74      0.78      0.76       2500
 weighted avg      0.85      0.83      0.84       2500
```

7.Model Comparison:

Metric	Tuned Random Forest	Tuned XGBoost	Best Choice
Accuracy	84.44%	83.44%	RF (Slightly higher overall)
F1 Score	0.6182	0.6208	XGBoost
Recall (Class 1 - Churn)	0.63 (63%)	0.68 (68%)	XGBoost (Key Metric)
True Positives (Detected Churns)	315 / 497	339 / 497	XGBoost (+24 extra churners caught)
False Negatives (Missed Churns)	182	158	XGBoost (Lower risk)

7.1 XGBoost model is selected for final Deployment.

Reasons:

- Higher Recall for Minority Class (68% vs 63%):
- In customer churn prediction, catching churning customers is the primary goal.
- XGBoost successfully detected 339 actual churners, whereas Random Forest only caught 315.
- That is 24 additional high-risk customers identified by XGBoost.
- Better Overall F1 Score (0.6208 vs 0.6182):
- XGBoost achieves a higher F1 score, demonstrating a superior trade-off between Precision and Recall for the imbalanced class.
- Fewer Missed Churners (False Negatives):
- XGBoost reduced missed churners down to 158 (compared to 182 in Random Forest), minimizing the business risk of losing undetected customers.

7.2 Final Conclusion:

- "Although Random Forest achieved a slightly higher raw accuracy (84.44% vs 83.44%), the Hyperparameter-Tuned XGBoost Classifier is selected for deployment.
- XGBoost demonstrated superior capability in detecting minority class instances, achieving a higher Recall of 68% (339 detected churners) and an overall higher F1-score of 0.6208, making it the most business-aligned model for churn prevention."

8. Deployment

8.1 Model Export

Description:

The best-performing model (Tuned XGBoost), feature scaler, and column structure were serialized and exported into pickle (.pkl) files using the joblib library for production deployment.

Files Exported:

- `xgb_model.pkl`: Saved Tuned XGBoost Classifier object (`best_xgb`).
- `scaler.pkl`: Saved fitted StandardScaler object for transforming incoming raw data.
- `model_columns.pkl`: Saved feature column names to ensure strict input ordering during inference.

```
[122] ✓ 0.0s import joblib
      Generate + Code + Markdown Python

[123] ✓ 1.3s joblib.dump(scaler, 'scaler.pkl')
      Python
... ['scaler.pkl']

[124] ✓ 1.2s joblib.dump(best_xgb, 'xgb_model.pkl')
      print("model saved successfully")
      Python
... model saved successfully

[125] ✓ 0.0s joblib.dump(x_train.columns.tolist(), "model_columns.pkl")
      print("columns saved successfully")
      Python
... columns saved successfully
```

8.2 API Development

Technology Used:

- FastAPI (Python framework for building APIs)
- Uvicorn (ASGI server to run FastAPI)
- Pydantic (Data validation using Python type annotations)

API Endpoint:

POST /predict

8.3 API Implementation Code (app.py)

```
app.py > predict_churn
1  from fastapi import FastAPI
2  from pydantic import BaseModel
3  import joblib
4  import pandas as pd
5  import numpy as np
6
7  app=FastAPI(title="Bank Customer Churn Prediction API")
8
9  model=joblib.load("xgb_model.pkl")
10 scaler=joblib.load("scaler.pkl")
11 model_columns=joblib.load("model_columns.pkl")
12
13 class CustomerData(BaseModel):
14     Credit_Score:int
15     Age:int
16     Tenure:int
17     Balance:float
18     NumOfProducts:int
19     HasCreditCard:int # 1 or 0
20     IsActiveMember:int # 1 or 0
21     EstimatedSalary:float
22     Country:str #options:'France','Germany','Spain'
23     Gender:str #options:'Male','Female'
24
25 @app.get("/")
26 def home():
27     return {"message": "Bank Customer Churn Prediction API is running"}
28
29 @app.post("/predict")
30 def predict_churn(data:CustomerData):
31
32
33     if data.Country == "Germany":
34
35         Country_Germany = 1
36         Country_Spain = 0
37
38     elif data.Country == "Spain":
39         Country_Germany = 0
40         Country_Spain = 1
41     else:
42
43         Country_Germany = 0
44         Country_Spain = 0
45
46     Gender_male = 1 if data.Gender == "Male" else 0
47 ..
```

```

48 | input_data = {
49 |     "Credit_Score": data.Credit_Score,
50 |     "Age": data.Age,
51 |     "Tenure": data.Tenure,
52 |     "Balance": data.Balance,
53 |     "NumOfProducts": data.NumOfProducts,
54 |     "HasCreditCard": data.HasCreditCard,
55 |     "IsActiveMember": data.IsActiveMember,
56 |     "EstimatedSalary": data.EstimatedSalary,
57 |     "Country_Germany": Country_Germany,
58 |     "Country_Spain": Country_Spain,
59 |     "Gender_Male": Gender_male
60 | }
61
62 | num_df=pd.DataFrame([
63 |     "Credit_Score": data.Credit_Score,
64 |     "Age": data.Age,
65 |     "Tenure": data.Tenure,
66 |     "Balance": data.Balance,
67 |     "NumOfProducts": data.NumOfProducts,
68 |     "HasCreditCard": data.HasCreditCard,
69 |     "IsActiveMember": data.IsActiveMember,
70 |     "EstimatedSalary": data.EstimatedSalary
71 | ])
72 | scaled_df=pd.DataFrame(scaler.transform(num_df),columns=num_df.columns)
73
74 | cat_df=pd.DataFrame([
75 |     "Country_Germany": Country_Germany,
76 |     "Country_Spain": Country_Spain,
77 |     "Gender_Male": Gender_male
78 | ])
79
81 | df=pd.concat([scaled_df,cat_df],axis=1)
82 | df=df[model_columns]
83
84 | prediction = model.predict(df)[0]
85 | return {
86 |     "prediction": int(prediction),
87 |     "status": "Churned " if prediction == 1 else "Not churned",
88 | }
89

```

Output:

```

> TERMINAL
❏ PS D:\Entri\MachineLearning> uvicorn app:app --reload
INFO: Stopping reloader process [15792]
● PS D:\Entri\MachineLearning> & "C:\Users\venkatesan g\AppData\Local\Programs\Python\Python314\python.exe" d:/Entri/MachineLearning/
app.py
○ PS D:\Entri\MachineLearning> uvicorn app:app --reload
INFO: Will watch for changes in these directories: ['D:\Entri\MachineLearning']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [6920] using StatReload
INFO: Started server process [20548]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:60863 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:55949 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:55949 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:59246 - "POST /predict HTTP/1.1" 200 OK
WARNING: StatReload detected changes in 'app.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [20548]
INFO: Started server process [23276]
INFO: Waiting for application startup.
INFO: Application startup complete.

```

API:

The screenshot shows the API documentation for the "Bank Customer Churn Prediction API". The URL in the browser is `http://127.0.0.1:8000/docs#/default/predict_churn_predict_post`. The API version is `0.1.0` and the OpenAPI version is `OAS 3.1`. The documentation is for the "default" API. The "POST /predict" endpoint is highlighted, with the description "Predict Churn". The "Parameters" section is empty, with "No parameters" listed. The "Request body" is marked as "required" and the media type is set to "application/json". There are "Cancel" and "Reset" buttons. At the bottom, there are "Edit Value" and "Schema" links.

Bank Customer Churn Prediction API `0.1.0` `OAS 3.1`

`/openapi.json`

default

GET / Home

POST /predict Predict Churn

Parameters

No parameters

Request body *required*

application/json

Edit Value | Schema

8.4 Sample Request:

The screenshot shows the same API documentation page as above, but with a sample request body entered in the "Request body" field. The sample request is a JSON object with the following fields: "Credit_Score", "Age", "Tenure", "Balance", "NumOfProducts", "HasCreditCard", "IsActiveMember", "EstimatedSalary", "Country", and "Gender". The "Execute" button is highlighted in blue.

POST /predict Predict Churn

Parameters

No parameters

Request body *required*

application/json

Edit Value | Schema

```
{
  "Credit_Score": 500,
  "Age": 78,
  "Tenure": 1,
  "Balance": 5632,
  "NumOfProducts": 1,
  "HasCreditCard": 0,
  "IsActiveMember": 1,
  "EstimatedSalary": 58695,
  "Country": "Germany",
  "Gender": "Male"
}
```

Execute Clear

8.5 Sample Response:

The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:8000/docs#/default/predict_churn_predict_post`. The main content area is titled "Responses" and contains the following sections:

- Curl:** A code block showing a curl command to POST a JSON payload to `http://127.0.0.1:8000/predict`. The payload includes fields like `Credit_Score`, `Age`, `Tenure`, `Balance`, `NumOfProducts`, `HasCreditCard`, `IsActiveMember`, `EstimatedSalary`, `Country`, and `Gender`.
- Request URL:** A text field showing `http://127.0.0.1:8000/predict`.
- Server response:** A table with two columns: "Code" and "Details".
 - Code:** 200
 - Details:**
 - Response body:** A JSON object: `{ "prediction": 0, "status": "Not churned" }`. There is a "Download" button next to it.
 - Response headers:** `content-length: 39`, `content-type: application/json`, `date: Thu, 23 Jul 2026 10:59:56 GMT`, and `server: uvicorn`.

9. Conclusion

9.1 Final Findings

- **Data Preprocessing & Scaling:** Categorical variables (Geography, Gender) were encoded using One-Hot Encoding (`drop_first=True`), and numerical features were standardized using `StandardScaler` to prevent feature dominance.
- **Model Evaluation & Tuning:** Baseline models were evaluated across various metrics. Tuned XGBoost Classifier outperformed Random Forest and other baselines, achieving an Accuracy of 83.44%, an F1-Score of 0.6208, and a Recall of 68% (0.68).
- **Handling Imbalance & Trade-offs:** Experiments with SMOTE showed that algorithmic hyperparameter tuning without synthetic oversampling provided cleaner decision boundaries, achieving higher precision and better overall F1-performance.

- **Model Selection: Tuned** XGBoost successfully identified 339 out of 497 churned customers (+24 more true churners than Random Forest), making it the optimal choice for deployment.

9.2 Business Impact

- **Proactive Customer Retention:** By identifying 68% of potential churners in advance, the bank's marketing and customer relationship teams can target high-risk clients with tailored incentives, loyalty rewards, or revised credit terms before they leave.
- **Cost Reduction:** Retaining existing customers is significantly cheaper than acquiring new ones. Reducing false negatives (missed churners) directly saves marketing budgets and minimizes revenue loss.
- **Production Readiness:** The final model and scaler were serialized (xgb_model.pkl, scaler.pkl) and deployed via a lightweight FastAPI REST interface, allowing seamless integration into the bank's existing CRM systems for real-time risk scoring.

Future Improvements

- **Additional Data Collection:** Incorporate longitudinal/time-series behavioural data, such as recent transaction frequency, customer service interaction logs, and feedback scores, to capture temporal churn signals.
- **Advanced Algorithms & Deep Learning:** Experiment with advanced boosting techniques like CatBoost or deep learning architectures (e.g., TabNet) specialized for tabular data.
- **Cloud Deployment:** Deploy the FastAPI application to scalable cloud platforms such as AWS Elastic Beanstalk, GCP, or Azure behind a load balancer.
- **Monitoring & MLOps:** Set up continuous monitoring tools (e.g., Evidently AI, Prometheus) to track data drift, model performance decay, and latency over time.
- **Automated Model Retraining Pipeline:** Implement an automated CI/CD and retraining workflow (using tools like Airflow or Kubeflow) to regularly retrain the model on fresh monthly data.